

Alejandro Gallardo Marqués

José Pablo De La Haba Márquez

Contenedores Docker vs VM

Javier Castilla Arroyo

Alejandro Gómez Amaro

Índice

➤ 1. Introducción

➤ 3. VM (Ubuntu Server)

➤ 5. Análisis Comparativo

➤ 2. Contenedor Docker (Nginx)

➤ 4. Metodología Evaluación

➤ 6. Conclusiones



1. Introducción

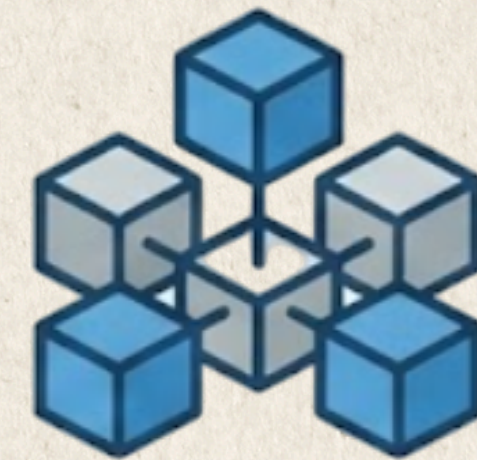
¿VM o Contenedores?



Servidores
Físicos
Monolíticos



Virtualización
(VM)



Contenedorización
(Docker)



El dilema: ¿Eficiencia o agilidad o aislamiento total?



Objetivo del Proyecto: Comparativa de rendimiento real (Benchmarking) entre Docker y Ubuntu Server (VM)

Entorno De Pruebas



Servicio: Servidor Web Nginx (mismo servicio en ambos entornos)



Herramientas de Estrés: Apache Benchmark (ab) + Script de automatización en Python.

Métricas Analizadas:

- Latencia y Throughput (Peticiones/Segundo)
- Consumo de Hardware (CPU y RAM)
- Validación Estadística (T-Student)





2. Contenedor Docker (Nginx)

2. Contenedor Docker (Nginx)



¿Qué es Docker?

Docker es una plataforma de código abierto que permite automatizar el despliegue de aplicaciones dentro de contenedores de software.

- Virtualización a nivel del SO
- Kernel compartido



¿Qué es Nginx?

Servidor web que vamos a desplegar con Docker

- Código abierto y alto rendimiento
 - Soporta cargas de tráfico muy elevadas
 - Arquitectura asíncrona y orientada a eventos.
-

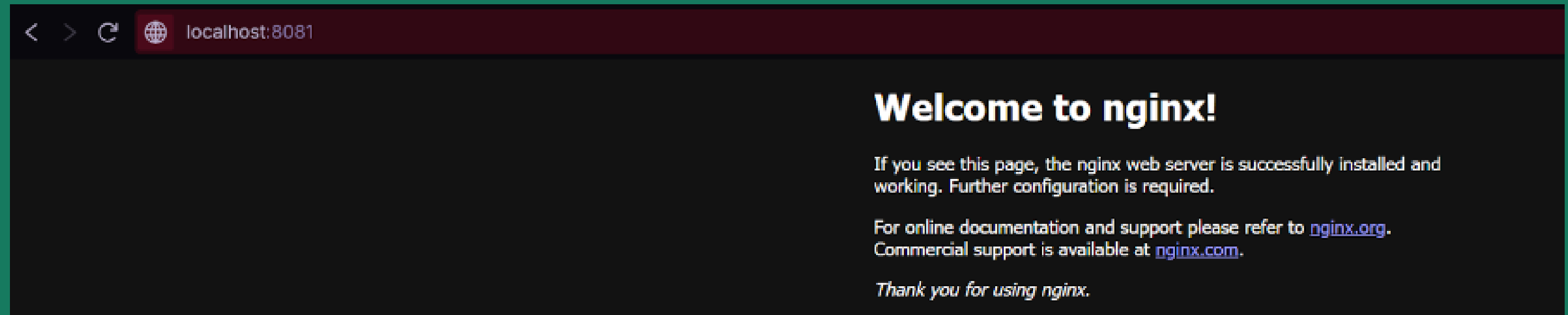
Inicialización de Nginx

1. Instalar imagen de Nginx

```
docker pull nginx
```

2. Levantar el servicio

```
docker run --name servidor-nginx -d -p 8081:80 nginx
```



Docker Desktop

 **docker.desktop** PERSONAL

Ask Gordon BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Containers [Give feedback](#)

Container CPU usage ⓘ
0.00% / 3200% (32 CPUs available)

Container memory usage ⓘ
26.5MB / 14.15GB

Show charts



☐ Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last st	Actions
☐	 servidor-nginx	1f4bf9563807	nginx	8081:80 	0%	43 min	  

Walkthroughs

 **Multi-container applications**
8 mins

```
$ docker init
```



Containerize your application
3 mins

[View more in the Learning center](#)

 Engine running

RAM 1.41 GB CPU 0.00% Disk: 1.58 GB used (limit 1006.85 GB)

  v4.52.0



3. VM(Ubuntu Server)

3. VM (Ubuntu Server)



¿Qué es una VM?

Una máquina virtual es una emulación por software de un sistema informático físico que funciona como un entorno aislado.

Hipervisor (como una VirtualBox) para abstraer y asignar recursos de hardware (CPU, RAM, disco) del equipo anfitrión

Incluye su propio kernel independiente, librerías y aplicaciones

Trabajamos con Oracle VM VirtualBox

Configuración de las dependencias de VM

sudo apt update && sudo apt install -y nginx sysstat apache2-utils openssh-server python3

```
cesi@ubuntu-server-cesi:~$ sudo systemctl status ssh
• ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: active (running) since Tue 2025-12-02 16:40:48 UTC; 39s ago
   TriggeredBy: • ssh.socket
     Docs: man:sshd(8)
           man:sshd_config(5)
   Process: 3236 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 3237 (sshd)
     Tasks: 1 (limit: 4583)
    Memory: 3.1M (peak: 4.0M)
       CPU: 198ms
    CGroup: /system.slice/ssh.service
            └─3237 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Dec 02 16:40:48 ubuntu-server-cesi systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Dec 02 16:40:48 ubuntu-server-cesi sshd[3237]: Server listening on 0.0.0.0 port 22.
Dec 02 16:40:48 ubuntu-server-cesi sshd[3237]: Server listening on :: port 22.
Dec 02 16:40:48 ubuntu-server-cesi systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
Dec 02 16:41:10 ubuntu-server-cesi sshd[3245]: Accepted password for cesi from 10.0.2.2 port 3924 ssh2
Dec 02 16:41:10 ubuntu-server-cesi sshd[3245]: pam_unix(sshd:session): session opened for user cesi(uid=1000) by cesi(uid=0)
cesi@ubuntu-server-cesi:~$ sudo systemctl status nginx
• nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Tue 2025-12-02 16:27:14 UTC; 14min ago
     Docs: man:nginx(8)
   Main PID: 2375 (nginx)
     Tasks: 3 (limit: 4583)
    Memory: 2.4M (peak: 5.3M)
       CPU: 69ms
    CGroup: /system.slice/nginx.service
            └─2375 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
              └─2377 "nginx: worker process"
                └─2378 "nginx: worker process"

Dec 02 16:27:14 ubuntu-server-cesi systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Dec 02 16:27:14 ubuntu-server-cesi systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
cesi@ubuntu-server-cesi:~$ _
```

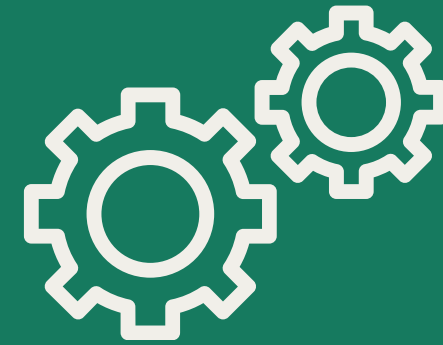



4. Metodología Evaluación

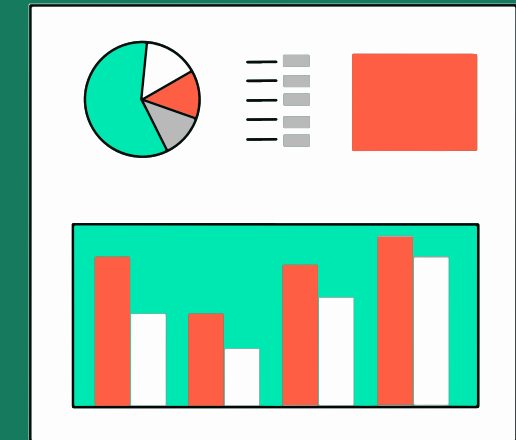
4. Metodología Evaluación



Pruebas ejecutadas bajo el mismo nivel de hardware y condiciones de Red.

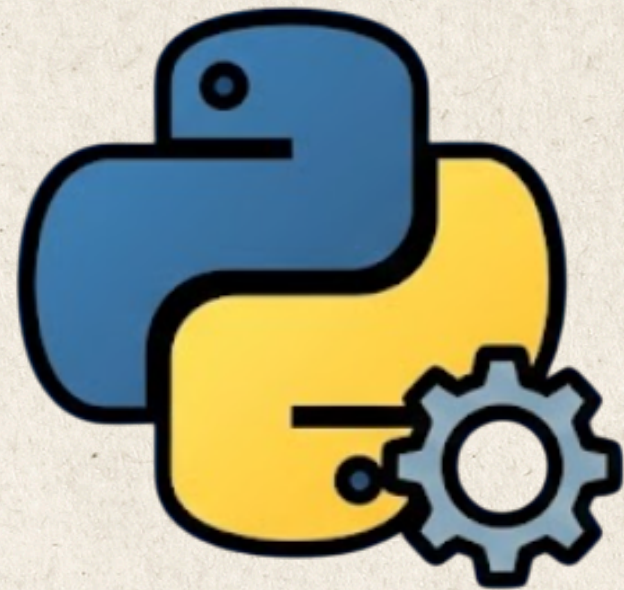


Desarrollo de scripts en Python para eliminar el error humano.

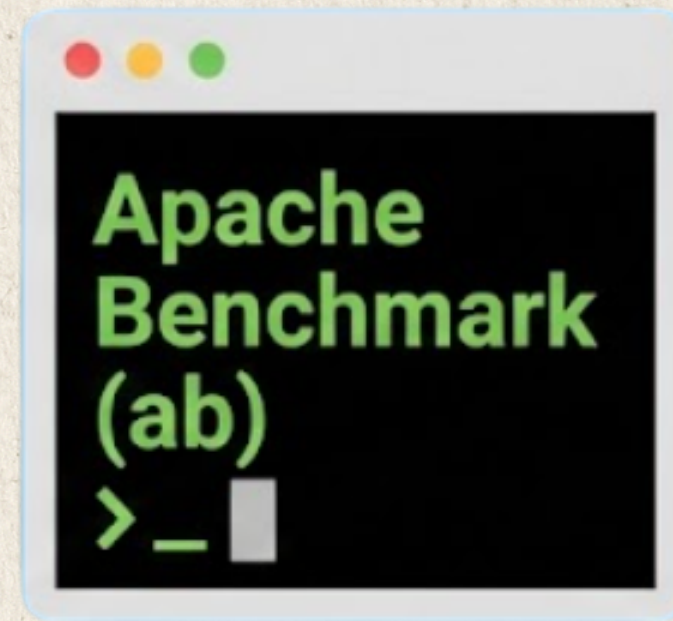


Análisis de Latencia, Throughput (Req/s) y Consumo de Recursos (CPU y RAM)

4.1 Arquitectura de Medición



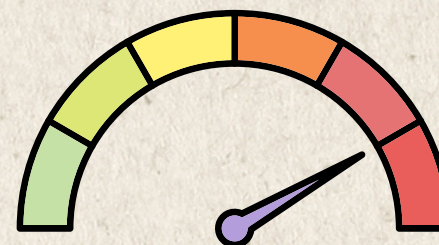
Orquestador:
Script propio en
Python (benchmark.py)



Motor de Carga
Apache Benchmark (ab)
Estándar de la industria



Monitorización
Docker Stats
(contenedores) vs htop
(VM)




```
TARGET_PORT = 8081

# Dirección del servidor (localhost)
HOST = "127.0.0.1"

# URL completa
URL = f"http://{HOST}:{TARGET_PORT}/"

# Niveles de concurrencia (Usuarios simultáneos)
# Empezamos suave y terminamos fuerte
CONCURRENCY_LEVELS = [10, 50, 100, 200, 500]

# Cuántas peticiones totales se hacen en cada prueba
REQUESTS_PER_TEST = 2000

# Nombre del archivo donde se guardarán los datos
OUTPUT_FILE = f"resultados_benchmark_puerto_{TARGET_PORT}.csv"
# =====

def check_ab_installed():
    """Comprueba si Apache Benchmark está instalado/accesible"""
    try:
        subprocess.run(["ab", "-V"], capture_output=True, check=False)
        return True
    except FileNotFoundError:
        print("❌ ERROR: No se encuentra el comando 'ab' (Apache Benchmark).")
        return False

def run_benchmark(concurrency, requests):
    """Ejecuta una prueba de carga y devuelve los datos"""
    print(f"🚀 Lanzando prueba: {requests} peticiones con {concurrency} usuarios simultáneos...")

    # Comando ab: -n (total) -c (concurrentes) -k (keep-alive) -q (quiet)
    # Nota: En Windows a veces hace falta poner 'ab.exe' explícitamente si no está en el PATH
    cmd = f"ab -n {requests} -c {concurrency} -k {URL}"

    try:
        # Ejecutamos el comando en la terminal del sistema
        result = subprocess.run(cmd, capture_output=True, text=True, shell=True)

        if result.returncode != 0:
            print(f"⚠️ Error en la ejecución de AB: {result.stderr}")
            return None

        output = result.stdout

        # --- EXTRACCIÓN DE DATOS (REGEX) ---
        # 1. Peticiones por segundo (Rendimiento puro)
        rps_match = re.search(r"Requests per second:\s+([\d\.]+)", output)
        rps = float(rps_match.group(1)) if rps_match else 0

        # 2. Tiempo por petición (Latencia media)
        lat_match = re.search(r"Time per request:\s+([\d\.]+\s+[\ms\]\s+(mean\)", output)
        latency = float(lat_match.group(1)) if lat_match else 0

        # 3. Peticiones fallidas (Errores)
        fail_match = re.search(r"Failed requests:\s+(\d+)", output)
        failed = int(fail_match.group(1)) if fail_match else 0

        print(f"✅ Resultado: {rps} Req/s | Latencia: {latency} ms | Fallos: {failed}")
        return [concurrency, rps, latency, failed]
```

4.2 Automatización de Pruebas (Scripting)



DEFINIMOS UN OBJETIVO (IP:PUERTO)



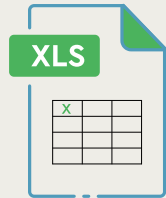
BUCLE DE CARGA (10 → 50 USUARIOS)



LANZAR APACHE BENCHMARK



CAPTURAR Y PARSEAR DATOS (REGEX)



EXPORTARA CSV

4.3 Escenarios de Carga Definidos

- 10-50 Usuarios

Tráfico Normal
(Navegación
Estándar)



- 100-200 Usuarios

Picos de Actividad
(Carga Alta)



- 500 Usuarios

Stress Test
(Punto de Saturación
/ Ataque)



Dato clave: Total de peticiones por ciclo: 2000





5. *Análisis Comparativo*

5.1 Docker

Tiempo de despliegue

```
PS C:\Users\User> Measure-Command { docker run --name servidor-nginx -d -p 8081:80 nginx }

Days           : 0
Hours          : 0
Minutes        : 0
Seconds        : 0
Milliseconds   : 854
Ticks          : 8547123
TotalDays      : 9,89250347222222E-06
TotalHours     : 0,000237420083333333
TotalMinutes   : 0,014245205
TotalSeconds   : 0,8547123
TotalMilliseconds : 854,7123
```

Tamaño de la imagen

```
C:\Users\User>docker system df

TYPE          TOTAL          ACTIVE        SIZE          RECLAIMABLE
Images        1              1             225MB         225MB (100%)
Containers    1              1             81.92kB       0B (0%)
Local Volumes 0              0             0B            0B
Build Cache   0              0             0B            0B
```


Sobrecarga del servidor

```
PS C:\Users\User\Desktop\UNIVERSITY\3º-Año\CESI\Trabajo> python benchmark.py
--- INICIANDO BENCHMARK CONTRA http://127.0.0.1:8081/ ---
🚀 Lanzando prueba: 2000 peticiones con 10 usuarios simultáneos...
✅ Resultado: 6645.27 Req/s | Latencia: 1.505 ms | Fallos: 0
🚀 Lanzando prueba: 2000 peticiones con 50 usuarios simultáneos...
✅ Resultado: 10585.09 Req/s | Latencia: 4.724 ms | Fallos: 0
🚀 Lanzando prueba: 2000 peticiones con 100 usuarios simultáneos...
✅ Resultado: 11371.58 Req/s | Latencia: 8.794 ms | Fallos: 0
🚀 Lanzando prueba: 2000 peticiones con 200 usuarios simultáneos...
✅ Resultado: 13103.93 Req/s | Latencia: 15.263 ms | Fallos: 0
🚀 Lanzando prueba: 2000 peticiones con 500 usuarios simultáneos...
✅ Resultado: 7720.49 Req/s | Latencia: 64.763 ms | Fallos: 0

🎉 ¡Terminado! Los datos están en el archivo: resultados_benchmark_puerto_8081.csv
Ahora abre ese archivo en Excel y haz las gráficas.
PS C:\Users\User\Desktop\UNIVERSITY\3º-Año\CESI\Trabajo>
```


Estado del servidor

The image shows a split-screen view. On the left is the Docker Desktop interface, and on the right is a Windows PowerShell terminal window.

Docker Desktop Interface:

- Top bar: "docker.desktop PERSONAL" with a "Sign in" button.
- Left sidebar: Navigation icons for Containers, Images, Clusters, Resources, Settings, and Help.
- Main area: "Containers" tab. It shows "Container CPU usage" at 6.19% / 3200% (32 CPUs available) and "Container memory usage" at 28.58MB / 14.15GB. A "Show charts" link is present.
- Below the usage stats is a search bar and a toggle for "Only running".
- A table lists the running containers:

	Name	Container ID	Image	Port(s)	Actions
☐	servidor-nginx	1f4b9563807	nginx	8081:80	
- Bottom right of the main area says "Showing 1 item".
- Bottom section: "Walkthroughs" with two cards:
 - "Multi-container applications" (8 mins)
 - "Containerize your application" (3 mins)
- Bottom status bar: "RAM 1.47 GB CPU 0.03% Disk: 1.58 GB used (lim: 1006.85 GB) v4.32.0".

Windows PowerShell Terminal:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas característi
cas y mejoras. https://aka.ms/PSWindows

PS C:\Users\User> cd .\Desktop\
PS C:\Users\User\Desktop> cd .\UNIVERSITY\
PS C:\Users\User\Desktop\UNIVERSITY> cd .\3º-Año\
PS C:\Users\User\Desktop\UNIVERSITY\3º-Año> cd .\CESI\
PS C:\Users\User\Desktop\UNIVERSITY\3º-Año\CESI> cd .\Trabajo\
PS C:\Users\User\Desktop\UNIVERSITY\3º-Año\CESI\Trabajo> python benchmark.py
--- INICIANDO BENCHMARK CONTRA http://127.0.0.1:8081/ ---
🔥 Lanzando prueba: 2000 peticiones con 10 usuarios simultáneos...
✅ Resultado: 21514.4 Req/s | Latencia: 0.465 ms | Fallos: 0
🔥 Lanzando prueba: 2000 peticiones con 50 usuarios simultáneos...
✅ Resultado: 23342.13 Req/s | Latencia: 2.142 ms | Fallos: 0
🔥 Lanzando prueba: 2000 peticiones con 100 usuarios simultáneos...
✅ Resultado: 21781.15 Req/s | Latencia: 4.608 ms | Fallos: 0
🔥 Lanzando prueba: 2000 peticiones con 200 usuarios simultáneos...
✅ Resultado: 21689.39 Req/s | Latencia: 9.221 ms | Fallos: 0
🔥 Lanzando prueba: 2000 peticiones con 500 usuarios simultáneos...
✅ Resultado: 1627.52 Req/s | Latencia: 307.215 ms | Fallos: 0
```


5.2 Ubuntu Server

```
cesi@ubuntu-server-cesi:~$ ls
benchmark.py
cesi@ubuntu-server-cesi:~$ python3 benchmark.py
--- INICIANDO BENCHMARK CONTRA http://127.0.0.1:80/ ---
* Lanzando prueba: 2000 peticiones con 10 usuarios simultáneos...
  * Resultado: 5082.06 Req/s | Latencia: 1.968 ms | Fallos: 0
* Lanzando prueba: 2000 peticiones con 50 usuarios simultáneos...
  * Resultado: 10434.49 Req/s | Latencia: 4.792 ms | Fallos: 0
* Lanzando prueba: 2000 peticiones con 100 usuarios simultáneos...
  * Resultado: 6638.52 Req/s | Latencia: 15.064 ms | Fallos: 0
* Lanzando prueba: 2000 peticiones con 200 usuarios simultáneos...
  * Resultado: 8686.89 Req/s | Latencia: 23.023 ms | Fallos: 0
* Lanzando prueba: 2000 peticiones con 500 usuarios simultáneos...
  * Resultado: 4844.56 Req/s | Latencia: 103.209 ms | Fallos: 0

* ¡Terminado! Los datos están en el archivo: resultados_benchmark_puerto_80.csv
Ahora abre ese archivo en Excel y haz las gráficas.
cesi@ubuntu-server-cesi:~$ _
```


Comparativa de latencia y peticiones

Docker

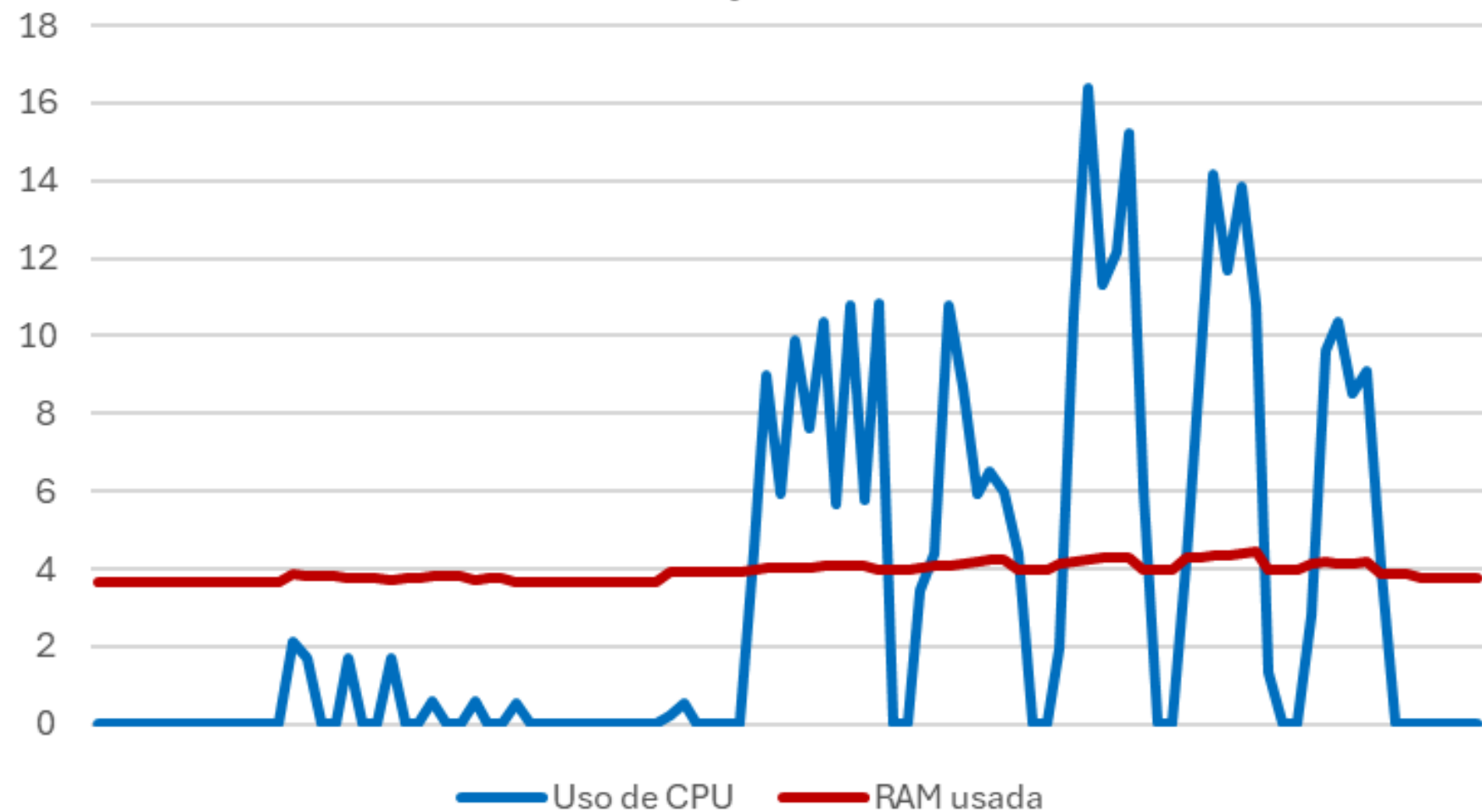
	A	B	C	D
	Usuarios_Simultaneos	Peticiones_por_Segundo	Latencia_Media_ms	Errores
2	10	4307.02	2.322	0
3	50	10867.97	4.601	0
4	100	11013.76	09.08	0
5	200	4543.85	44.016	0
6	500	5168.24	96.745	0

VM

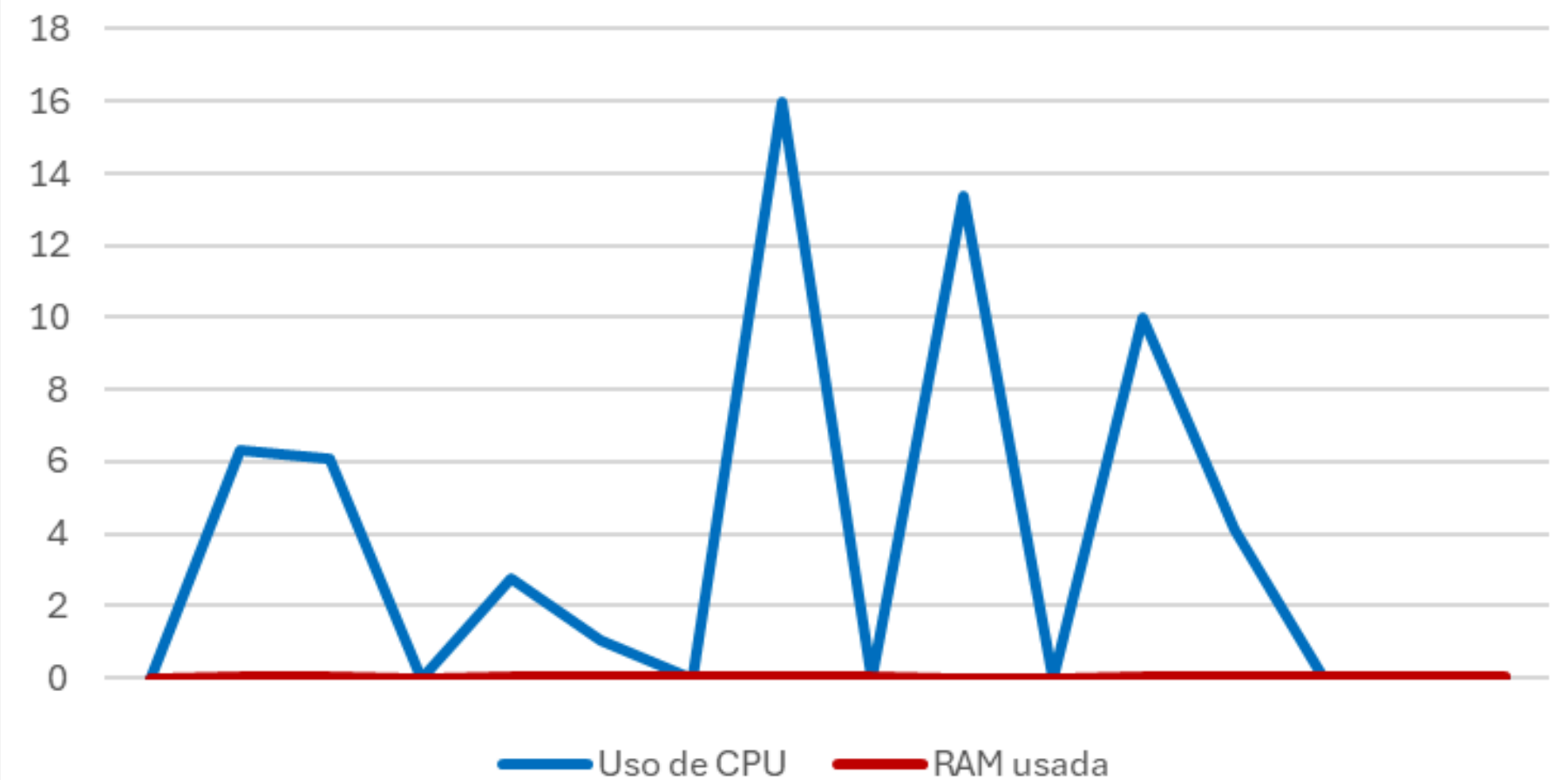
	A	B	C	D
	Usuarios_Simultaneos	Peticiones_por_Segundo	Latencia_Media_ms	Errores
2	10	5082.06	1.968	0
3	50	10434.49	4.792	0
4	100	6638.52	15.064	0
5	200	8686.89	23.023	0
6	500	4844.56	103.209	0

Comparativa de Consumo de CPU y RAM

Uso de CPU y RAM en VM



Uso de CPU y RAM en Docker





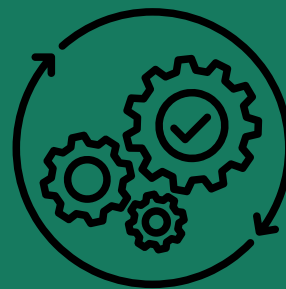
6. Conclusiones

Veredicto Final: La Superioridad Estructural De Docker



Rendimiento Bruto

- Doble de Capacidad: +104% de peticiones/segundo frente a la VM
- Estabilidad bajo estrés: Latencia de 52 ms constante vs degradación VM.



- Modelo Elástico: Uso de recursos bajo demanda (~20MB)
- Modelo Estático: La VM bloquea 4 GB permanente



- Validado: diferencias confirmadas por test T-Student.
- Confianza: 95% de certeza de que Docker es superior



Para el despliegue de servicios web modernos, la máquina virtual tradicional es una solución ineficiente. **Docker** se establece como el **estándar óptimo**, combinando máximo rendimiento con **mínimo consumo de recursos**.

Gracias
por su atención